Cloud-Native Monitoring System Demo

Kubernetes · Prometheus · Grafana

Author: Paras Khosla
Date: 10-12-25

Github code repository: https://github.com/paraskhosla/cloud-native-monitoring-system

---

Overview

This project demonstrates a cloud-native monitoring system for a containerized FastAPI application.

It shows how metrics are generated, collected, and visualized using:

- Kubernetes (deployment)
- Prometheus (metrics collection)
- Grafana (visualization)

Step 1: System Architecture

The system follows this pipeline:
User -> FastAPI -> Kubernetes -> Prometheus -> Grafana

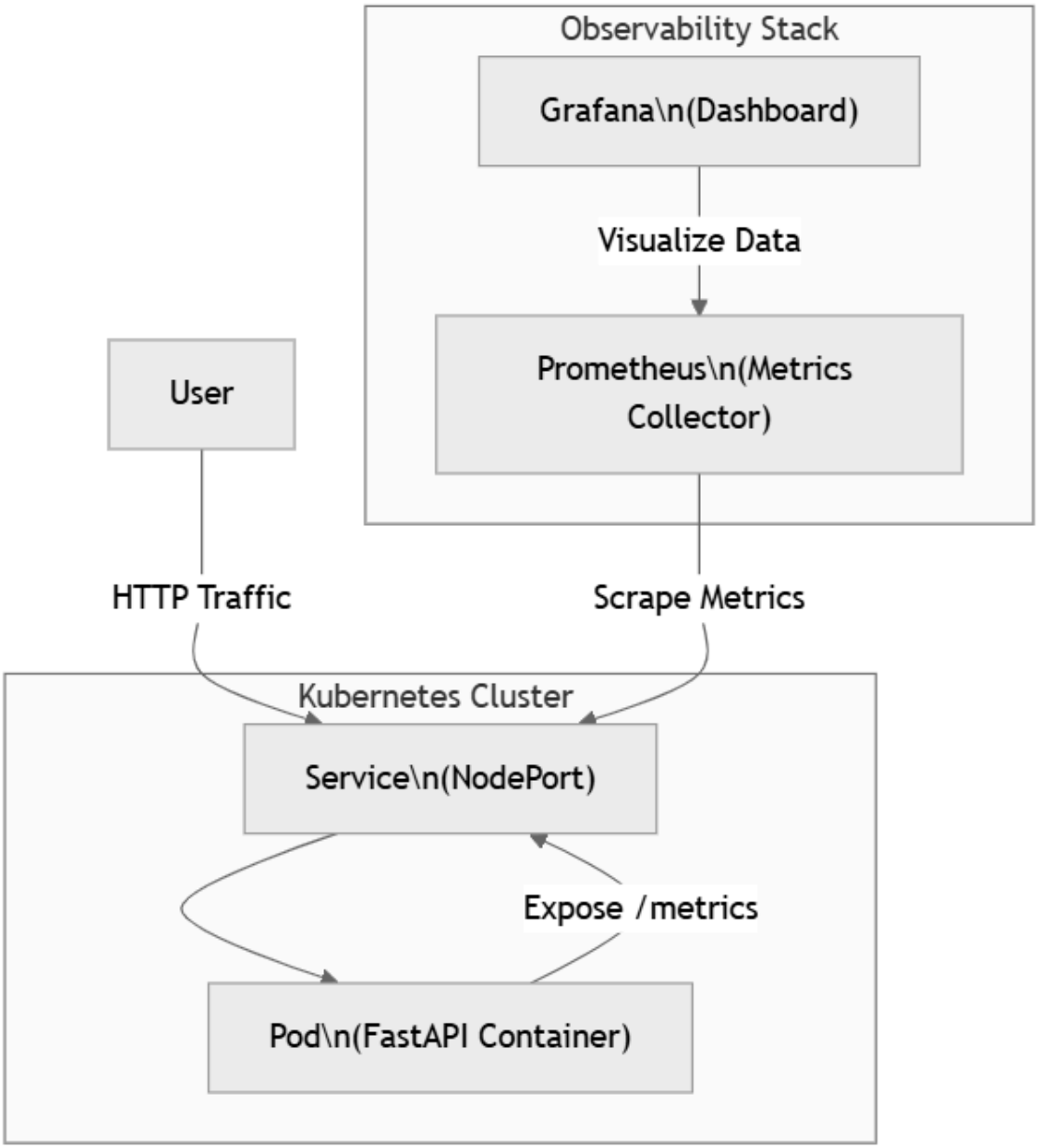

Figure 1: Architecture of the monitoring system

Step 2: Application with Metrics Endpoint

The FastAPI application exposes metrics using the /metrics endpoint.

```
    return { data : "Here is your data" }

# @app.get("/metrics")
# def metrics():
#     return generate_latest()

@app.get("/metrics")
def metrics():
    return Response(generate_latest(), media_type="text/plain")
```

Figure 2: Metrics endpoint implementation in FastAPI

📡 Step 3: Running Application & Metrics Output

Once the application is running, metrics are available via:
http://localhost:8000/metrics

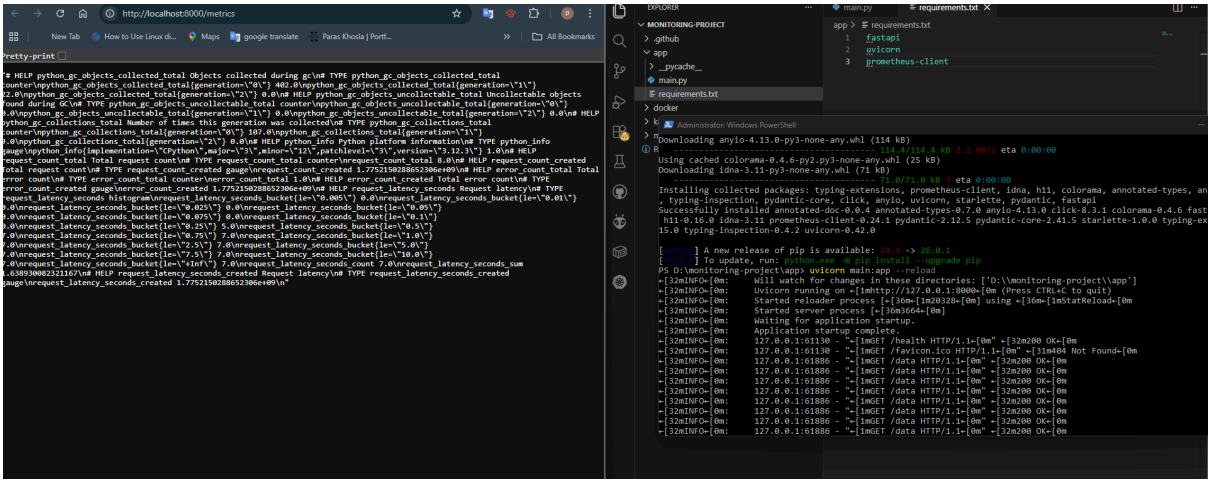

Figure 3: Raw Prometheus metrics exposed by the application

Step 4: Kubernetes Deployment

The application is deployed into a Kubernetes cluster using:

- Deployment
- Service (NodePort)

```
Administrator: Windows PowerShell
service/grafana-service created
deployment.apps/prometheus unchanged
service/prometheus-service unchanged
configmap/prometheus-config unchanged
PS D:\monitoring-project> kubectl apply -f monitoring/
deployment.apps/grafana unchanged
service/grafana-service unchanged
deployment.apps/prometheus unchanged
service/prometheus-service unchanged
configmap/prometheus-config unchanged
PS D:\monitoring-project> kubectl get pods
NAME                              READY   STATUS              RESTARTS   AGE
grafana-5fddfc6dd-2z9sz           0/1     ContainerCreating   0          34s
monitoring-app-67d6948c98-ksc54   1/1     Running             0          10m
prometheus-5dfb9956fd-hql9m       1/1     Running             0          33m
PS D:\monitoring-project> kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
grafana-5fddfc6dd-2z9sz           1/1     Running   0          119s
monitoring-app-67d6948c98-ksc54   1/1     Running   0          11m
prometheus-5dfb9956fd-hql9m       1/1     Running   0          34m
PS D:\monitoring-project> minikube service grafana-service --url
http://127.0.0.1:65512
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

Figure 4: Application running inside Kubernetes cluster

📊 Step 5: Prometheus Scraping Metrics

Prometheus is configured to scrape metrics from the Kubernetes service.

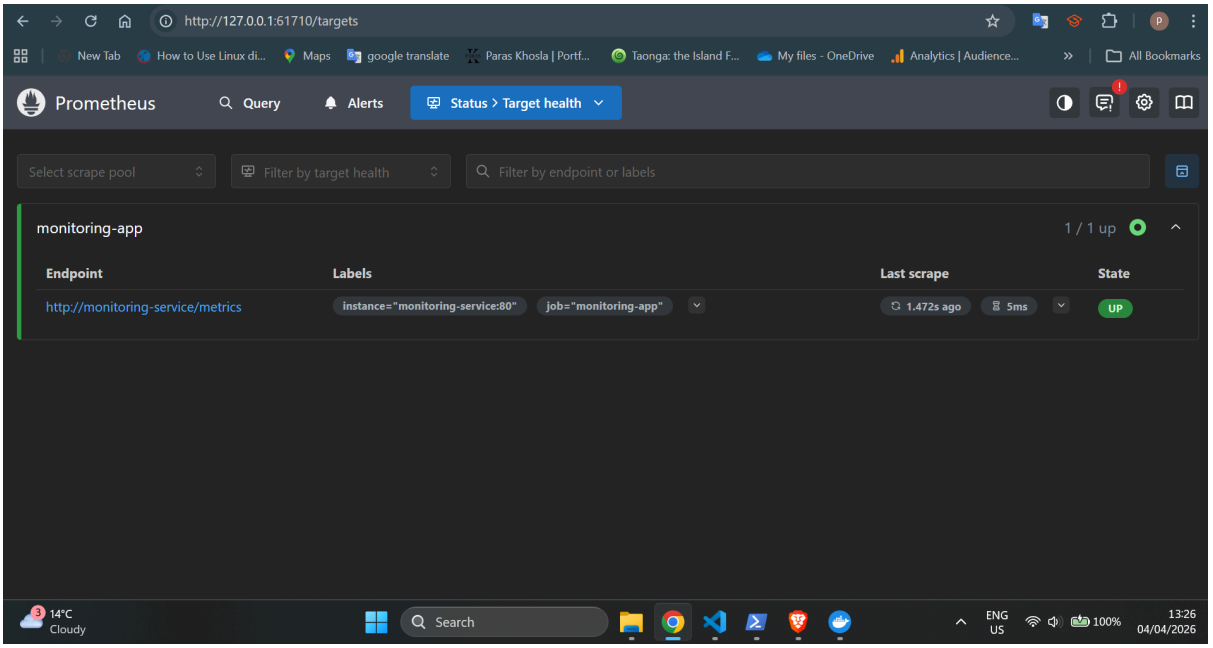

Figure 5: Prometheus successfully scraping metrics

Step 6: Grafana Dashboard

Grafana connects to Prometheus and visualizes the metrics.
Dashboard includes:

- Request rate
- CPU usage
- Memory usage
- Latency

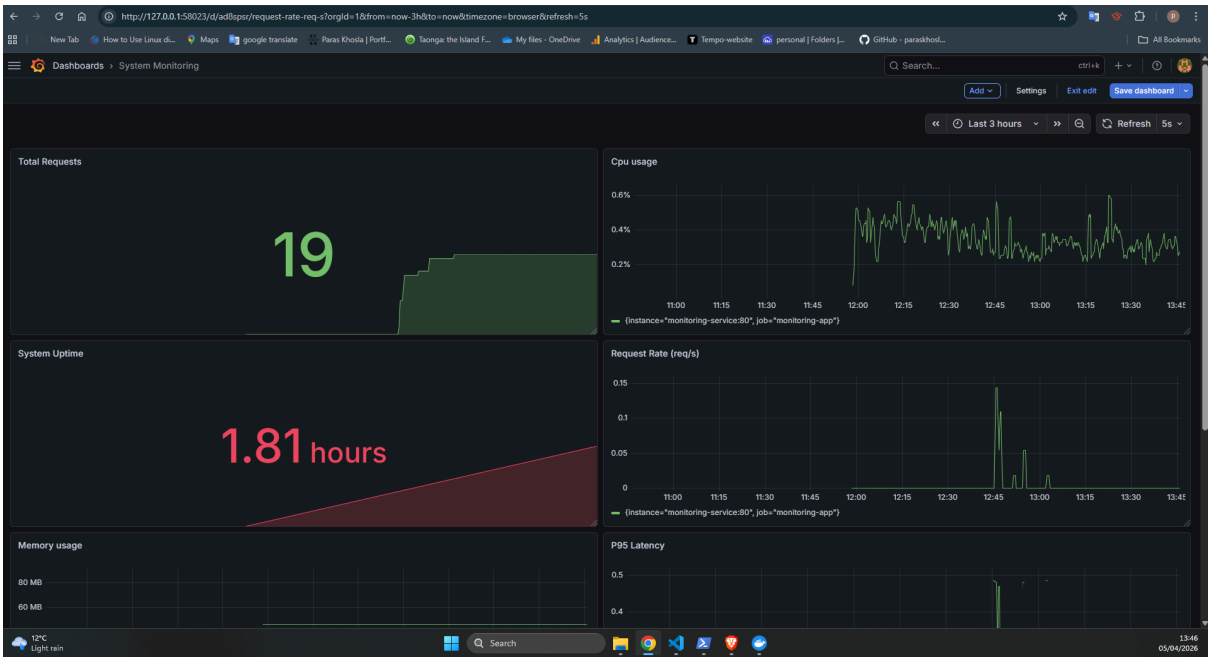

Figure 6: Grafana dashboard with real-time monitoring

Step 7: Container Environment

The system runs inside Docker containers with Kubernetes (Minikube).

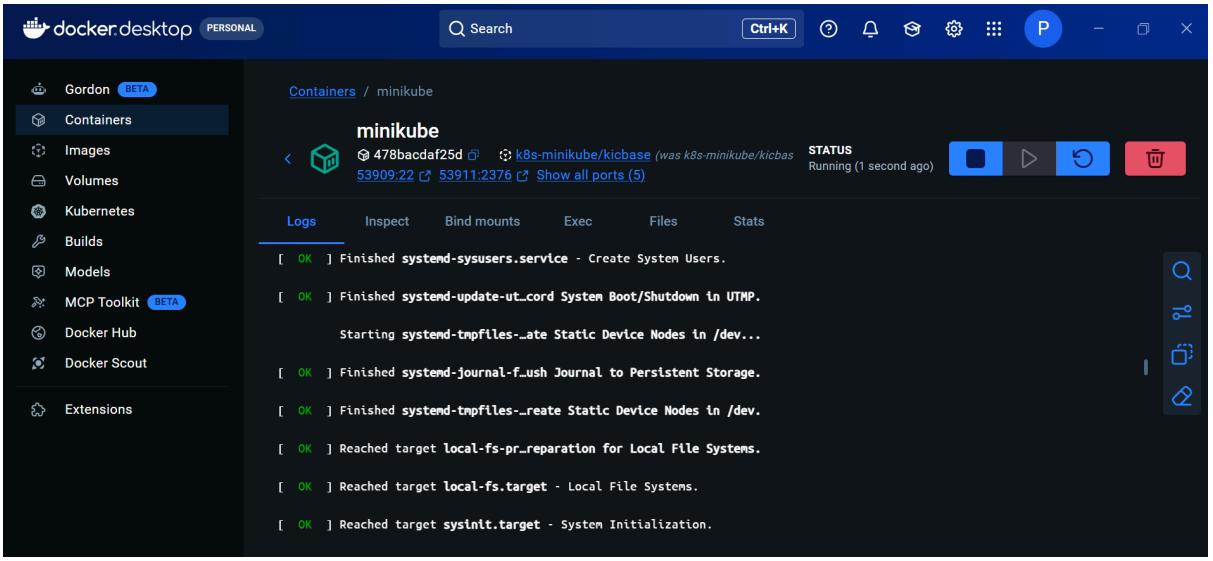

Figure 7: Containers running the monitoring stack

Final Result

The system provides an end-to-end observability pipeline:

- Metrics generated by application
- Collected by Prometheus
- Visualized in Grafana